AgeOverflow

Software Architecture

Semester 1, 2026

Richard Thomas & Millie Hughes

Warning

This document is a draft and is subject to change.

Summary

In this assignment, you will demonstrate your ability to *design*, *implement*, and *deploy* a web API that can process a high load, i.e. a scalable application. You are to deploy an API to analyse images of headshots to identify an individual's generation cohort. Specially your application needs to support:

- Analysing images received via an API request.
- Providing access to a specified REST API, e.g. for use by front-end interfaces and other applications.
- Remaining responsive while analysing images.
- Reporting statistics about the analysed images.

Your service will be deployed to AWS and will undergo automated correctness and load-testing to ensure it meets the requirements.

Info

The scenario in this document is purely fictional for the purposes of this assignment and does not represent the views or opinions of the authors.

1 Introduction

For this assignment, you are working for AgeOverflow, a new UQ start-up. AgeOverflow uses (fictional) machine learning techniques. The analysis is able to identify percentage certainty of the given assets to their generational cohort, with an age approximation.

Task AgeOverflow uses a microservices architecture to implement their analysis platform. The CTO saw on your resume that you are taking Software Architecture and has assigned you to design and implement the generation cohort analysis service. This service must scale to cope with the anticipated large number of requests.

Requirements Automated identification of generational cohorts is an important service to ensure legal compliance in Australia, and potentially other countries. Manual verification by personnel is labour intensive and time consuming. Automated analysis provides faster responses with varying degrees of certainty, to allow manual review only when its required. This is critical as tens or hundreds of thousands of analyses need to be performed daily for users of new services.

© The University of Queensland 2026

AgeOverflow's Generation Cohort Analysis Service (GCAS) needs to be designed to scale to match demand. Clients will obtain photographic documentation from users. These images will be sent to the GCAS for analysis.

The algorithms used to analyse the images are computationally intensive. It is not possible to return a result immediately for a submitted image set. Clients, will need to query the GCAS to obtain results at a later time. Results can be queried for a single verification, or a batch of verifications for a client or individual.

As clients have a significant backlog of users to verify, the service must be able to provide results in a timely manner. Delayed verification can put a range of our clients under legal scrutiny.

Persistence is an important characteristic of the platform. Resubmitting analysis requests due to lost data would place unnecessary legal and data risk on clients, at times when they may be under extreme pressure to deliver results to federal authorities. Upon receiving an analysis request, and after error checking, the GCAS must guarantee that the data has been saved to persistent storage before returning a success response.

2 Interface

As you are operating in a microservices context, other service providers have been given an API specification for your service. They have been developing their services based on this specification so you must match it *exactly*.

The interface specification is available to all service owners online:
https://csse6400.uqcloud.net/assessment/cloud/

3 Implementation

The following constraints apply to the implementation of your assignment solution.

3.1 Analysis Engine

You have been provided with a command line tool called `AgeOverflow-Engine` that must be used to analyse sample image sets. The tool has varying performance, due to the difficulty of identifying cohorts. You will have to work around this bottleneck in the design and development of the GCAS.

Your service **must** utilise the provided `AgeOverflow-Engine` command line tool provided for this assignment. The compiled binaries are available in the tool's GitHub repository:
https://github.com/CSSE6400/AgeOverflow-Engine.

Warning

You are **not** allowed to reimplement or modify this tool.

The analysis engine requires pre-processing of the images. This is done by an upstream service. For testing purposes, you **must** use the sample images provided in the analysis engine's repository[1]. If you try to generate your own images, they are likely to fail analysis or give false results.

3.2 AWS Services

Please make note of the AWS services[2] that you can use in the AWS Learner Lab, and the limitations that are placed on the usage of these services. The most up-to-date list of available services, and restrictions

[1] https://github.com/CSSE6400/AgeOverflow-Engine
[2] https://csse6400.uqcloud.net/resources/LearnerLab-Restrictions.pdf

on their usage, is described in the Learner Lab `Readme` tab.

3.3 External Services

You may **not** use services or products from outside of the AWS Learner Lab environment. For example, you may not host instances of the `engine` command line tool on another cloud platform (e.g. Google Cloud).

You may **not** use services or products that run on AWS infrastructure external to your Learner Lab environment. For example, you may not deploy a third-party product like MongoDB Atlas on AWS and then use it from your service.

You may **not** deploy machine learning or GPU backed services.

3.4 Scaling

A simplistic approach to manage high volumes of requests or high processing demands, is to have many compute nodes running to cater for the demand. Stage 3 of this assignment assesses your ability to scale your application to meet demand. This means your application needs to scale up ***and*** down to be economically efficient. From an evaluation perspective, this means

- starting your application with many compute nodes available and idle,
- scaling up faster than demand requires,
- or not scaling down in a timely manner,

will limit your final grade for stage 3. See the criteria on page 10 for details.

4 Submission

This assignment has three submissions.

1. March 27th – API Functionality
2. April 13th – Deployed to Cloud
3. May 1st – Scalable Application

All submissions are due at 15:00 on the specified date. Your solution for each submission must be committed and pushed to the GitHub repository specified in Section 4.3.

Each submission is to be in its **own branch**. You ***must*** use the branch names ***exactly*** as indicated below. Failure to use these branch names may result in your submission not being marked, and you obtaining a grade of X for the submission.

- **stage-1** for API Functionality, due on March 27th
- **stage-2** for Deployed to Cloud, due on April 13th
- **stage-3** for Scalable Application, due on May 1st

When marking a stage, we will checkout the branch for that stage. Any code in the main branch, or *any* other branch, will be ignored when marking. We will checkout the latest commit in the branch being marked. If the commit date and time is after the submission deadline, late penalties will be applied, unless you have an extension. Late penalties are described in the course profile[3].

[3] https://course-profiles.uq.edu.au/course-profiles/CSSE6400-21413-7620#assessment-detail-0

Note: Experience has shown that the large majority of students who make a late submission, lose more marks from the late penalty than they gain from any improvements they make to their solution. We *strongly* encourage you to submit your work on-time.

You should commit and push your work to your repository regularly. If a misconduct case is raised about your submission, a history of regular progress on the assignment through a series of commits could support your argument that the work was your own.

Extension requests **must** be made *prior* to the submission deadline via my.UQ[4].

Your repository **must** contain everything required to successfully deploy your application.

4.1 API Functionality Submission

Your first submission **must** include all of the following in your repository:

- Docker image (Dockerfile) of your implementation of the service API, including the source code and a mechanism to build and run the service.[5]
- A `local.sh` script that can be used to build and run your service locally. This script **must** be in the *root* directory of your repository. The `local.sh` script must launch your container with port 8080 being passed from the container to the testing environment, and your service **must** be available at `http://localhost:8080/api/v1/`.

We will run a suite of tests against your API at this URL.

4.2 Deployed to Cloud & Scalability Submissions

The second and third submissions **must** include all of the following in your repository:

- Your implementation of the service API, including the source code and a mechanism to build the service.[6]
- Terraform code that provisions your service in a fresh AWS environment.

When deploying your second and third submissions to mark, we will follow reproducible steps, outlined below. You may re-create the process yourself.

1. Your Git repository will be cloned locally.
2. The submission branch will be checked out.
3. If a `before.sh` file exists in the root directory, it will be run.
4. We will run `terraform init`, `terraform plan`, and `terraform apply -auto-approve` in the root directory of your project.
5. We will then run `terraform output -raw api > api.txt`. The result of this must be that `api.txt` contains the URL at which your API is deployed (e.g. `http://my-api.com/api/v1/` or `http://123.231.213.012/api/v1/`). The `api.txt` file must **only** contain the URL. Do not include ***any*** other text in the output.
6. Automated functionality and load tests will be run against the URL discovered in the `api.txt` file.

[4] `https://my.uq.edu.au/`

[5] If you use external libraries, ensure that you pin the versions to avoid external changes breaking your application.

[6] If you use external libraries, ensure that you pin the versions to avoid external changes breaking your application.

Warning

Ensure your service does not exceed the resource limits of AWS Learner Labs. If you use up your allocated budget in the Learner Lab, you will not be able to run any services. **Note:** AWS will **deactivate** your account if it has twenty or more concurrently running instances.

4.3 GitHub Repository

You will be provisioned with a private repository on GitHub for this assignment, via GitHub Classroom.[7] You must associate your GitHub username with your UQ student ID in the Classroom.

Associating your GitHub username with another student's ID, or getting someone else to associate their GitHub username with your student ID, is academic misconduct[8].

If for some reason you have accidentally associated your GitHub username with the wrong student ID, contact the course staff as soon as possible.

4.4 Folder Structure

Your Github repository should look similar to the following, at a minimum:

```
/
├── .github
│   └── workflows
│       └── cloud.yml
├── .gitignore
├── README.md
├── refs.md
├── AI.md
├── AI-log.md
├── local.sh
├── deploy.sh
└── ...
```

4.5 Tips

Terraform plan/apply hanging If your `terraform plan` or `terraform apply` command hangs without any output, check your AWS credentials. Using credentials of an expired Learner Lab session will cause Terraform to hang.

Fresh AWS Learner Lab Your AWS Learner Lab can be reset using the reset button in the toolbar.

To ensure that you are not accidentally depending on anything specific to your Learner Lab environment, we recommend that you reset your lab prior to final submission. Note that resetting the lab can take a *considerable* amount of time, on the order of hours. You should do this at *least* 3 to 4 hours before the submission deadline. Please do not wait to the last minute.

[7] https://classroom.github.com/a/_o6vwSxE

[8] https://my.uq.edu.au/information-and-services/manage-my-program/student-integrity-and-conduct/academic-integrity-and-student-conduct

Deploying with Docker In this course, you have been shown how to use Docker containers to deploy on ECS. You may refer to the practical worksheets for a description of how to deploy with containers [1].

5 Criteria

Your assignment submission will be assessed on its ability to support the specified use cases. Testing is divided into functionality, deployment, and scalability testing, corresponding to the three submission stages of the assignment. Functionality testing is to ensure that your backend software and API meet the MVP requirements by satisfying the API specification without any excessive load. Deployment is to ensure that this MVP can then be hosted on the target cloud provider. Scalability testing is based upon several likely usage scenarios. The scenarios create different scaling requirements.

5.1 API Functionality

10%, from the 40% of the grade for the assignment, is for correctly implementing the API specification, irrespective of whether it is able to cope with high loads. A suite of automated API tests will assess the correctness of your implementation, via a sequence of API calls.

5.2 Deployed to Cloud

10%, from the 40% of the grade for the assignment, is for deploying a correctly implemented service to AWS, irrespective of whether it is able to cope with high loads. The deployment will be assessed by running a script that deploys your service to AWS and then runs a suite of automated API tests to assess the correctness of your implementation.

5.3 Scalable Application

20%, from the 40% of the grade for the assignment, will be derived from how well your service handles various scenarios. These scenarios will require you to consider how your application performs under different load characteristics. Examples of possible scenarios are described below. These are not descriptions of specific tests that will be run, rather they are examples of the types of tests that may be run.

Normal Circumstances Average number of analysis requests are submitted by a wide range of clients. These requests are distributed fairly evenly over the working day. Few of the analysis requests are urgent. Queries for results are also distributed fairly evenly over the day.

Curiosity Killed the Server A client creates a web portal allowing an auditor to view results for their users. Tens of thousands of requests will try to access their users' results in a short period of time.

Compliance Early Stages There is a significant increase in the number of analysis request submitted by a few clients. These clients also mark a much higher number of the requests as being urgent as they are new users. Other clients are operating as per normal circumstances. There are many more queries for results, often being repeated for any pending analysis jobs. Auditors make many requests for batches of results from clients or for users.

Compliance Mid Stages There are periods of time where several clients submit a large volume of analysis requests. There are other periods of time with reduced numbers of analysis requests. Up to 15% of the

requests may be given an urgent status. Queries for results follow the same fluctuating pattern as analysis requests. Auditors make a moderate number of requests for batches of results from clients or for users.

Compliance Peak Almost all clients submit a very large volume of analysis requests. Up to 30% of the requests may be given an urgent status. The service must still analyse non-urgent requests in a timely manner. If it does not, the risk is that clients will start to make all requests urgent. Due to the high volume of analysis requests, there will be a high volume of queries for results. Any results that are still pending analysis jobs, will be repeated. Auditors make a moderate number of requests for batches of results from almost all clients. Clients start querying batches of results for individual users.

Compliance Tail Almost all clients submit a large volume of analysis requests, but timing may be variable. Up to 10% of the requests may be given an urgent status. Due to the high volume of analysis requests, there will be a high volume of queries for results. Most requests for results that are still pending analysis jobs, will be repeated. Auditors make a moderate number of requests for batches of results from almost all clients. Clients query batches of results for a moderate number of users.

Auditing and BAU Some clients submit a large volume of analysis requests due to influx of new users or backlog. Up to 5% of the requests may be given an urgent status. Other clients are back to operating as per normal circumstances. There will be a moderate volume of queries for results. Most requests for results that are still pending analysis jobs, will be repeated. Auditors make a large number of requests for batches of results from almost all clients. Clients query batches of results for a large number of users.

Research Project An internal researcher wants to analyse the results of the analysis against a false positive database provided by the clients. They will query all the results from all the clients to generate their database of results. The system must remain responsive to analysis requests while processing the researcher's queries.

5.4 Marking

Persistence is a core functional requirement of your service. Your grade for the assignment will be capped at **4**, if your implementation does not save

- analysis requests, with their associated images, or
- analysis results to persistent storage.

Your persistence mechanism must be robust, so that it can cope with catastrophic failure of your service. If all running instances of your services are terminated, the system must be able to restart and guarantee that it has not lost *any* data about analysis requests for which it returned a success response to the client. It must also guarantee it has not lost any analysis results, after changing the status of analysis job from "pending".

There will not be a test that explicitly kills all services and restarts the system. This will be assessed based on the services you use and how your implementation invokes those services. Not saving data to a persistent data store, or returning a success response before the data has been saved, or not saving an analysis job result effectively, are the criteria that determine whether you have successfully implemented persistence.

Functionality of your service is worth **10%** from the 40% of the grade for the assignment. This is based on successful implementation of the given API specification and the ability to use the provided tool in your implementation.

Deploying your service is worth **10%** from the 40% of the grade for the assignment. This is based on the successful deployment, using Terraform, of your service to AWS and the ability to access the service via

the API. Your service must be fully functional while deployed, so the functionality tests can be run which determines the marks for deployment.

Scaling your application to successfully handle the usage scenarios accounts for the remaining **20%** of the grade for the assignment. The scenarios described in section 5.3 provide guidance as to the type of scalability issues your system is expected to handle. They are not literal descriptions of the exact loads that will be used. Tests related to scenarios that involve more complex behaviour will have higher weight than other tests.

The scenarios will evaluate whether your service is being wasteful in resource usage. The amount of resources deployed in your AWS account will be monitored to ensure that your service implements a scaling up ***and*** scaling down procedure.

All stages of the assessment will be marked using automation. A subset of the functionality tests will be released before the first submission deadline.

Please refer to the marking criteria at the end of this document.

6 Academic Integrity

As this is a higher-level course, you are expected to be familiar with the importance of academic integrity in general, and the details of UQ's rules. If you need a reminder, review the Academic Integrity Modules[9]. Submissions will be checked to ensure that the work submitted is not plagiarised or of no academic merit.

This is an *individual* assignment. You may **not** discuss details of approaches to solve the problem with other students in the course. As some students may have an extension and then decide to make a late submission, you may **not** discuss details of a submission stage with other students until **five weeks** *after* the original submission date. i.e. You may not discuss how to

- implement **functionality** until *after* **May 18**th;[10]
- **deploy** the service to the cloud until *after* **May 18**th; and
- implement **scalability** until *after* **June 5**th.

All code that you submit **must** be your own work. You may **not** directly copy code that you have found on-line to solve parts of the assignment. If you find ideas from on-line sources (e.g. Stack Overflow), you must cite and reference[11] these sources. Use the IEEE referencing style[12] for citations and references. Citations must be included in a comment at the location where the idea is used in your code. All references for citations **must** be included in a file called `refs.md`. This file must be in the root directory of your repository.

6.1 AI Usage

You are developing a moderately complex software system. It is expected that you will make use of generative AI tools (e.g. Copilot) to assist you in implementing and testing your solutions.

You **must** include, in the root directory of your repository, a file called `AI.md` that indicates the generative AI tools that you used, how you used them, and the extent of their use. (e.g. All code was written by providing Copilot with class descriptions and then revising the generated code. Classes A and B were produced by the following prompts to ChatGPT and were then adapted to work in the assignment's context.)

You **must** also include, in the root directory of your repository, a file called `AI-log.md` that contains a log of the history of your interaction with the AI tool(s) that you used while developing your solution.

[9] https://web.library.uq.edu.au/study-and-learning-support/coursework/academic-integrity-modules

[10] Students may improve their functionality for the stage-2 submission, consequently you may not discuss how to implement functionality until after this date.

[11] https://guides.library.uq.edu.au/referencing

[12] https://libraryguides.vu.edu.au/ieeereferencing/gettingstarted

Uncited or unreferenced material, or unacknowledged use of generative AI tools, will be treated as not being your own work. Significant amounts of cited or acknowledged work from other sources, will be considered to be of no academic merit.

References

[1] E. Hughes, B. Webb, and R. Thomas, "Database & container deployment," vol. 5 of *CSSE6400 Practicals*, The University of Queensland, March 2025. https://csse6400.uqcloud.net/practicals/week05.pdf.

Cloud-Infrastructure Assignment Criteria

Criteria	Standard						
	Exceptional (7)	**Advanced (6)**	**Proficient (5)**	**Functional (4)**	**Developing (3)**	**Little Evidence (2)**	**No Evidence (1)**
API Functionality 10%	API passes at least 85% of the full test suite, including all valid GET, POST & PUT requests, & persistence.	API passes at least 75% of the full test suite, including most valid GET requests with complex queries, all valid POST & PUT requests, & persistence.	API passes at least 65% of the full test suite, including some valid GET requests with complex queries, at least valid POST requests, & persistence.	API passes at least 50% of the full test suite.	API passes at least 40% of the full test suite.	API passes at least 20% of the full test suite.	API passes less than 20% of the full test suite.
Deployed to Cloud 10%	Deployed API passes at least 85% of the full test suite, including all valid GET, POST & PUT requests, & persistence.	Deployed API passes at least 75% of the full test suite, including most valid GET requests with complex queries, all valid POST & PUT requests, & persistence.	Deployed API passes at least 65% of the full test suite, including some valid GET requests with complex queries, at least valid POST requests, & persistence.	Deployed API passes at least 50% of the full test suite.	Deployed API passes at least 40% of the full test suite.	Deployed API passes at least 20% of the full test suite.	Deployed API passes less than 20% of the full test suite.
Scalable Application 20%	Nearly all complex scenarios are handled well. Resources have been used efficiently (scale up & down).	Most complex scenarios are handled well. Resources have been used efficiently (scale up & down).	A few complex scenarios are handled well, or resource usage is not efficient (scale up or down are implemented poorly).	Only simple scenarios are handled well. Or, resource usage is not efficient (scale down is not implemented). Or, high demand is handled by launching many instances at once.	Some simple scenarios have been handled well.	Minimal simple scenarios are handled.	No scenarios are handled.

© The University of Queensland 2026